

BSDVP ACADEMY

Data structures through C

1. Introduction to Data Structures in C

A **data structure** is a way of organizing, managing, and storing data in a computer so that it can be accessed and modified efficiently. In C, data structures help to handle large amounts of data systematically, making the program more efficient.

Why Use Data Structures?

1. **Efficient Data Management:** Storing and retrieving data quickly.
 2. **Optimized Memory Usage:** Reducing unnecessary memory wastage.
 3. **Easy Data Modification:** Inserting, deleting, and updating data efficiently.
 4. **Faster Searching and Sorting:** Improves performance in algorithms.
-

Types of Data Structures in C

1. Linear Data Structures

These structures store data in a sequential manner.

- **Arrays:** Fixed-size collection of elements of the same type stored in contiguous memory.
- **Linked Lists:** A collection of nodes where each node contains data and a pointer to the next node.
- **Stacks:** Follows LIFO (Last-In-First-Out) principle.
- **Queues:** Follows FIFO (First-In-First-Out) principle.

2. Non-Linear Data Structures

These structures allow multiple relationships between elements.

- **Trees:** Hierarchical data structure with nodes (Binary Trees, Binary Search Trees, AVL Trees).
 - **Graphs:** Collection of nodes (vertices) connected by edges.
-

Searching in C

Searching is the process of finding an element in a data structure.

Types of Searching Algorithms

1. Linear Search

- A simple searching technique that checks each element one by one.

BSDVP ACADEMY

- Best for small datasets.
- **Time Complexity:** $O(n)$

C Implementation:

```
#include <stdio.h>

int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key)
            return i;
    }
    return -1;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);
    int key = 30;

    int result = linearSearch(arr, n, key);
    if (result != -1)
        printf("Element found at index %d\n", result);
    else
        printf("Element not found\n");

    return 0;
}
```

2. Binary Search

- Works on sorted arrays.
- Divides the array into two halves and repeatedly searches in the relevant half.
- **Time Complexity:** $O(\log n)$

BSDVP ACADEMY

C Implementation:

```
#include <stdio.h>
```

```
int binarySearch(int arr[], int low, int high, int key) {  
    while (low <= high) {  
        int mid = low + (high - low) / 2;  
  
        if (arr[mid] == key)  
            return mid;  
        else if (arr[mid] < key)  
            low = mid + 1;  
        else  
            high = mid - 1;  
    }  
    return -1;  
}
```

```
int main() {  
    int arr[] = {10, 20, 30, 40, 50};  
    int n = sizeof(arr) / sizeof(arr[0]);  
    int key = 30;  
  
    int result = binarySearch(arr, 0, n - 1, key);  
    if (result != -1)  
        printf("Element found at index %d\n", result);  
    else  
        printf("Element not found\n");  
    return 0;  
}
```

BSDVP ACADEMY

Sorting in C

Sorting is the process of arranging elements in a specific order (ascending or descending).

Types of Sorting Algorithms

1. Bubble Sort

- Repeatedly swaps adjacent elements if they are in the wrong order.
- Time Complexity: $O(n^2)$ (worst-case)

C Implementation:

```
#include <stdio.h>
```

```
void bubbleSort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = 0; j < n - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
            }  
        }  
    }  
}
```

```
void printArray(int arr[], int n) {  
    for (int i = 0; i < n; i++)  
        printf("%d ", arr[i]);  
    printf("\n");  
}
```

```
int main() {  
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
```

BSDVP ACADEMY

```
int n = sizeof(arr) / sizeof(arr[0]);
```

```
bubbleSort(arr, n);
```

```
printf("Sorted array: ");
```

```
printArray(arr, n);
```

```
return 0;
```

```
}
```

2. Selection Sort

- Selects the smallest element and swaps it with the first unsorted element.
- Time Complexity: $O(n^2)$

C Implementation:

```
#include <stdio.h>
```

```
void selectionSort(int arr[], int n) {
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        int minIndex = i;
```

```
        for (int j = i + 1; j < n; j++) {
```

```
            if (arr[j] < arr[minIndex])
```

```
                minIndex = j;
```

```
        }
```

```
        int temp = arr[minIndex];
```

```
        arr[minIndex] = arr[i];
```

```
        arr[i] = temp;
```

```
    }
```

```
}
```

```
void printArray(int arr[], int n) {
```

```
    for (int i = 0; i < n; i++)
```

BSDVP ACADEMY

```
        printf("%d ", arr[i]);  
        printf("\n");  
    }  
  
int main() {  
    int arr[] = {64, 25, 12, 22, 11};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    selectionSort(arr, n);  
    printf("Sorted array: ");  
    printArray(arr, n);  
  
    return 0;  
}
```

3. Insertion Sort

- Builds the sorted array one element at a time.
- Time Complexity: $O(n^2)$

C Implementation:

```
#include <stdio.h>
```

```
void insertionSort(int arr[], int n) {  
    for (int i = 1; i < n; i++) {  
        int key = arr[i];  
        int j = i - 1;  
        while (j >= 0 && arr[j] > key) {  
            arr[j + 1] = arr[j];  
            j--;  
        }  
        arr[j + 1] = key;  
    }  
}
```

BSDVP ACADEMY

```
    }  
}  
  
void printArray(int arr[], int n) {  
    for (int i = 0; i < n; i++)  
        printf("%d ", arr[i]);  
    printf("\n");  
}  
  
int main() {  
    int arr[] = {12, 11, 13, 5, 6};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    insertionSort(arr, n);  
    printf("Sorted array: ");  
    printArray(arr, n);  
  
    return 0;  
}
```

4. Quick Sort

- Uses divide-and-conquer by selecting a pivot and partitioning the array.
- Time Complexity: $O(n \log n)$ (best-case)

5. Merge Sort

- Recursively divides the array into halves and merges sorted halves.
 - Time Complexity: $O(n \log n)$
-

BSDVP ACADEMY

2. Data Structures in C: Linked Storage Representation (Linked Lists)

Introduction to Linked Storage Representation

In C, data structures can be stored in two primary ways:

1. **Sequential (Array-based) Storage**
2. **Linked Storage Representation (Linked Lists)**

Linked storage representation overcomes the limitations of arrays (such as fixed size) by dynamically allocating memory at runtime. The fundamental concept behind this is **pointers**, which enable efficient memory utilization.

Linked List: A Dynamic Data Structure

A **linked list** is a data structure consisting of nodes where each node contains:

1. **Data** (actual value)
2. **Pointer** (address of the next node in memory)

Unlike arrays, linked lists do not require contiguous memory allocation. Nodes are created dynamically as needed, making them efficient for insertions and deletions.

Types of Linked Lists

There are three main types of linked lists:

1. **Singly Linked List** – Each node points to the next node.
 2. **Doubly Linked List** – Each node has two pointers: one pointing to the next node and another to the previous node.
 3. **Circular Linked List** – The last node points back to the first node.
-

1. Singly Linked List (SLL)

A **Singly Linked List (SLL)** consists of nodes connected sequentially where each node stores:

- A **data field**
- A **pointer to the next node**
- The **last node points to NULL**, indicating the end of the list.

BSDVP ACADEMY

Structure Definition in C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Definition of a Node
```

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

```
// Function to create a new node
```

```
struct Node* createNode(int value) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = value;  
    newNode->next = NULL;  
    return newNode;  
}
```

```
// Function to insert a node at the beginning
```

```
void insertAtBeginning(struct Node** head, int value) {  
    struct Node* newNode = createNode(value);  
    newNode->next = *head;  
    *head = newNode;  
}
```

```
// Function to insert a node at the end
```

```
void insertAtEnd(struct Node** head, int value) {  
    struct Node* newNode = createNode(value);  
    if (*head == NULL) {
```

BSDVP ACADEMY

```
*head = newNode;

return;

}

struct Node* temp = *head;
while (temp->next != NULL)
    temp = temp->next;
temp->next = newNode;
}

// Function to delete a node
void deleteNode(struct Node** head, int key) {
    struct Node* temp = *head, *prev = NULL;
    if (temp != NULL && temp->data == key) {
        *head = temp->next;
        free(temp);
        return;
    }
    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) return;
    prev->next = temp->next;
    free(temp);
}

// Function to display the linked list
void displayList(struct Node* head) {
    while (head != NULL) {
```

BSDVP ACADEMY

```
printf("%d -> ", head->data);  
head = head->next;  
}  
printf("NULL\n");  
}
```

// Driver Code

```
int main() {  
    struct Node* head = NULL;  
    insertAtBeginning(&head, 10);  
    insertAtEnd(&head, 20);  
    insertAtEnd(&head, 30);  
    displayList(head);  
    deleteNode(&head, 20);  
    displayList(head);  
    return 0;  
}
```

Key Operations in Singly Linked List

1. **Insertion** – At the beginning, end, or a specific position.
2. **Deletion** – By value or position.
3. **Traversal** – Displaying the list contents.

2. Doubly Linked List (DLL)

A **Doubly Linked List (DLL)** consists of nodes where each node stores:

- A data field
- A pointer to the next node
- A pointer to the previous node

BSDVP ACADEMY

Structure Definition in C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Definition of a Node
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
    struct Node* prev;
```

```
};
```

```
// Function to create a new node
```

```
struct Node* createNode(int value) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = value;
```

```
    newNode->next = NULL;
```

```
    newNode->prev = NULL;
```

```
    return newNode;
```

```
}
```

```
// Function to insert a node at the beginning
```

```
void insertAtBeginning(struct Node** head, int value) {
```

```
    struct Node* newNode = createNode(value);
```

```
    if (*head != NULL) (*head)->prev = newNode;
```

```
    newNode->next = *head;
```

```
    *head = newNode;
```

```
}
```

```
// Function to insert a node at the end
```

BSDVP ACADEMY

```
void insertAtEnd(struct Node** head, int value) {  
    struct Node* newNode = createNode(value);  
    if (*head == NULL) {  
        *head = newNode;  
        return;  
    }  
    struct Node* temp = *head;  
    while (temp->next != NULL)  
        temp = temp->next;  
    temp->next = newNode;  
    newNode->prev = temp;  
}
```

// Function to display the linked list forward

```
void displayList(struct Node* head) {  
    while (head != NULL) {  
        printf("%d <-> ", head->data);  
        head = head->next;  
    }  
    printf("NULL\n");  
}
```

// Driver Code

```
int main() {  
    struct Node* head = NULL;  
    insertAtBeginning(&head, 10);  
    insertAtEnd(&head, 20);  
    insertAtEnd(&head, 30);  
    displayList(head);  
}
```

BSDVP ACADEMY

```
    return 0;
}
```

Key Operations in Doubly Linked List

1. **Insertion** – At the beginning, end, or a specific position.
 2. **Deletion** – From the beginning, end, or by value.
 3. **Traversal** – Forward and backward navigation.
-

3. Circular Linked List (CLL)

A **Circular Linked List (CLL)** is similar to SLL and DLL but with a circular structure:

- **Singly Circular List:** The last node points to the first node.
- **Doubly Circular List:** The last node points to the first node and vice versa.

Structure Definition in C (Singly Circular List)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Definition of a Node
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
};
```

```
// Function to insert a node at the end
```

```
void insertAtEnd(struct Node** head, int value) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = value;
```

```
    newNode->next = *head;
```

```
    if (*head == NULL) {
```

```
        *head = newNode;
```

BSDVP ACADEMY

```
newNode->next = *head;

} else {

    struct Node* temp = *head;

    while (temp->next != *head)

        temp = temp->next;

    temp->next = newNode;

}

}

// Function to display the circular linked list

void displayList(struct Node* head) {

    struct Node* temp = head;

    if (head == NULL) return;

    do {

        printf("%d -> ", temp->data);

        temp = temp->next;

    } while (temp != head);

    printf("(Back to Head)\n");

}

// Driver Code

int main() {

    struct Node* head = NULL;

    insertAtEnd(&head, 10);

    insertAtEnd(&head, 20);

    insertAtEnd(&head, 30);

    displayList(head);

    return 0;

}
```

BSDVP ACADEMY

Key Operations in Circular Linked List

1. **Insertion & Deletion** – Efficient because of circular pointers.
2. **Traversal** – Wraps around when reaching the last node.

3. Linked Lists in C

A **linked list** is a linear data structure where elements are stored in nodes. Each node contains two parts:

1. **Data** – Stores the actual data.
2. **Pointer (next)** – Stores the address of the next node.

Unlike arrays, linked lists use dynamic memory allocation, allowing flexibility in size modification without memory wastage.

Types of Linked Lists

1. **Singly Linked List (SLL)** – Each node has a pointer to the next node.
2. **Doubly Linked List (DLL)** – Each node has pointers to both the next and previous nodes.
3. **Circular Linked List (CLL)** – The last node points back to the first node, forming a loop.

1. Singly Linked List (SLL)

A **Singly Linked List** consists of nodes where each node contains:

- **Data**
- **Pointer to next node**

Structure of a Node in C

```
struct Node {  
    int data;  
    struct Node* next;  
};
```


BSDVP ACADEMY

Operations on Singly Linked List

(i) Insertion

- At the Beginning
- At the End
- At a Specific Position

(ii) Deletion

- From the Beginning
- From the End
- From a Specific Position

(iii) Traversal

- Accessing and printing elements.

Implementation of Singly Linked List

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Node structure
```

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

```
// Function to insert at the beginning
```

```
void insertAtBeginning(struct Node** head, int newData) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = newData;  
    newNode->next = *head;  
    *head = newNode;  
}
```

BSDVP ACADEMY

// Function to insert at the end

```
void insertAtEnd(struct Node** head, int newData) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = newData;  
    newNode->next = NULL;  
  
    if (*head == NULL) {  
        *head = newNode;  
        return;  
    }  
  
    struct Node* temp = *head;  
    while (temp->next != NULL) {  
        temp = temp->next;  
    }  
    temp->next = newNode;  
}
```

// Function to delete from beginning

```
void deleteFromBeginning(struct Node** head) {  
    if (*head == NULL) {  
        printf("List is empty\n");  
        return;  
    }  
    struct Node* temp = *head;  
    *head = (*head)->next;  
    free(temp);  
}
```

BSDVP ACADEMY

```
// Function to display linked list
void displayList(struct Node* node) {
    while (node != NULL) {
        printf("%d -> ", node->data);
        node = node->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    struct Node* head = NULL;

    insertAtBeginning(&head, 10);
    insertAtBeginning(&head, 20);
    insertAtEnd(&head, 30);
    insertAtEnd(&head, 40);

    printf("Linked List: ");
    displayList(head);

    deleteFromBeginning(&head);
    printf("After deletion: ");
    displayList(head);

    return 0;
}
```

BSDVP ACADEMY

2. Doubly Linked List (DLL)

A **Doubly Linked List** consists of nodes where each node contains:

- **Data**
- **Pointer to the next node**
- **Pointer to the previous node**

Structure of a Node in C

```
struct Node {  
    int data;  
    struct Node* prev;  
    struct Node* next;  
};
```

Operations on Doubly Linked List

- Insert, Delete, Traverse in both directions.

Implementation of Doubly Linked List

```
#include <stdio.h>  
#include <stdlib.h>
```

```
// Node structure
```

```
struct Node {  
    int data;  
    struct Node* prev;  
    struct Node* next;  
};
```

```
// Function to insert at the beginning
```

```
void insertAtBeginning(struct Node** head, int newData) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = newData;  
    newNode->next = *head;
```

BSDVP ACADEMY

```
newNode->prev = NULL;
```

```
if (*head != NULL)
```

```
    (*head)->prev = newNode;
```

```
*head = newNode;
```

```
}
```

```
// Function to display doubly linked list
```

```
void displayList(struct Node* node) {
```

```
    while (node != NULL) {
```

```
        printf("%d <-> ", node->data);
```

```
        node = node->next;
```

```
    }
```

```
    printf("NULL\n");
```

```
}
```

```
// Main function
```

```
int main() {
```

```
    struct Node* head = NULL;
```

```
    insertAtBeginning(&head, 10);
```

```
    insertAtBeginning(&head, 20);
```

```
    printf("Doubly Linked List: ");
```

```
    displayList(head);
```

```
    return 0;
```

```
}
```

BSDVP ACADEMY

3. Circular Linked List (CLL)

A **Circular Linked List** is a linked list where:

- **Last node points back to the first node.**
- Can be **Singly Circular** or **Doubly Circular**.

Implementation of Circular Linked List

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Node structure
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
};
```

```
// Function to insert a node
```

```
void insert(struct Node** head, int newData) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = newData;
```

```
    newNode->next = *head;
```

```
    struct Node* temp = *head;
```

```
    if (*head != NULL) {
```

```
        while (temp->next != *head)
```

```
            temp = temp->next;
```

```
        temp->next = newNode;
```

```
    } else {
```

```
        newNode->next = newNode;
```

```
    }
```

BSDVP ACADEMY

```
*head = newNode;

}

// Function to display Circular Linked List
void displayList(struct Node* head) {
    struct Node* temp = head;
    if (head != NULL) {
        do {
            printf("%d -> ", temp->data);
            temp = temp->next;
        } while (temp != head);
    }
    printf(" (Back to Head)\n");
}

// Main function
int main() {
    struct Node* head = NULL;

    insert(&head, 10);
    insert(&head, 20);
    insert(&head, 30);

    printf("Circular Linked List: ");
    displayList(head);

    return 0;
}
```

BSDVP ACADEMY

Advantages of Linked Lists

- **Dynamic Size:** Unlike arrays, linked lists can grow or shrink dynamically.
- **Efficient Insertions/Deletions:** Inserting or deleting a node is easier than in an array.
- **Memory Utilization:** No memory wastage due to fixed size.

Disadvantages of Linked Lists

- **More Memory Usage:** Requires extra space for pointers.
- **Slower Access:** Cannot perform direct access like arrays.
- **Complex Implementation:** More complex than arrays.

When to Use Linked Lists?

- When frequent insertions/deletions are needed.
- When memory size is unpredictable.
- When a dynamic and flexible data structure is required.

4. Linear Data Structures in C

Linear data structures are one of the fundamental types of data structures where elements are arranged in a sequential manner. In these structures, elements are stored in a linear order, making traversal straightforward (i.e., from one element to the next in a predictable way).

Types of Linear Data Structures

Linear data structures include:

1. **Arrays**
2. **Linked Lists**
3. **Stacks**
4. **Queues**

Each of these has its own advantages, disadvantages, and use cases.

1. Arrays

An **array** is a collection of elements stored at contiguous memory locations. Each element in an array can be accessed using an index.

Characteristics of Arrays:

- Fixed size (declared at compile time).

BSDVP ACADEMY

- Stored in contiguous memory locations.
- Elements are accessed using an index.
- Efficient random access ($O(1)$ time complexity).
- Insertion and deletion are costly ($O(n)$ in the worst case).

Declaration and Initialization in C:

```
#include <stdio.h>
```

```
int main() {  
    int arr[5] = {10, 20, 30, 40, 50}; // Declaration and initialization  
    printf("Element at index 2: %d\n", arr[2]); // Accessing elements  
    return 0;  
}
```

Operations on Arrays

1. Insertion

- Inserting an element at a specific position requires shifting elements.

```
void insertElement(int arr[], int *size, int pos, int value) {  
    for (int i = *size; i > pos; i--) {  
        arr[i] = arr[i - 1];  
    }  
    arr[pos] = value;  
    (*size)++;  
}
```

2. Deletion

- Deleting an element also requires shifting elements.

```
void deleteElement(int arr[], int *size, int pos) {  
    for (int i = pos; i < *size - 1; i++) {  
        arr[i] = arr[i + 1];  
    }  
    (*size)--;
```

BSDVP ACADEMY

```
}
```

2. Linked List

A **linked list** consists of nodes, where each node contains:

- **Data**
- **Pointer** to the next node

Unlike arrays, linked lists **do not have fixed memory allocation**, making them dynamic in nature.

Types of Linked Lists

1. **Singly Linked List** (Each node points to the next)
2. **Doubly Linked List** (Each node has pointers to both the next and previous nodes)
3. **Circular Linked List** (Last node connects to the first)

Structure of a Node in C:

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

3. Stack

A **stack** follows the **Last In, First Out (LIFO)** principle, meaning the last element inserted is the first to be removed.

Operations on Stack

- **Push** (Insert an element)
- **Pop** (Remove an element)
- **Peek/Top** (View the top element)
- **isEmpty** (Check if stack is empty)

Implementation of Stack Using Arrays

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX], top = -1;
```

BSDVP ACADEMY

```
void push(int value) {
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = value;
}

int pop() {
    if (top == -1) {
        printf("Stack Underflow\n");
        return -1;
    }
    return stack[top--];
}

int peek() {
    if (top == -1) {
        printf("Stack is empty\n");
        return -1;
    }
    return stack[top];
}

int main() {
    push(10);
    push(20);
    push(30);
    printf("Top element: %d\n", peek());
}
```

BSDVP ACADEMY

```
printf("Popped element: %d\n", pop());  
printf("Top element: %d\n", peek());  
return 0;  
}
```

4. Queues in Data Structures

Introduction to Queues

A **queue** is a linear data structure that follows the **First In, First Out (FIFO)** principle. This means that the first element inserted into the queue will be the first one to be removed, just like a real-world queue (e.g., a line at a supermarket checkout).

Characteristics of a Queue:

- **FIFO (First In, First Out):** The first element added is the first to be removed.
- **Insertion (Enqueue) occurs at the rear, while deletion (Dequeue) occurs at the front.**
- **Fixed size in arrays, dynamic in linked lists.**
- **Efficient for scheduling and buffering tasks.**

Types of Queues

There are several types of queues, each with different characteristics and use cases:

1. Simple Queue (Linear Queue)

- Follows FIFO strictly.
- Once the queue is full, no more insertions can be made even if there are deletions.

2. Circular Queue

- A more efficient version of a simple queue.
- Uses a circular array to make use of the empty spaces left by dequeued elements.

3. Double-Ended Queue (Deque)

- Allows insertion and deletion from both the front and rear.

4. Priority Queue

- Elements are dequeued based on priority instead of the order of arrival.

BSDVP ACADEMY

Queue Operations

1. **Enqueue** (Insertion at the rear)
 2. **Dequeue** (Deletion from the front)
 3. **Front** (Peek the front element)
 4. **isEmpty** (Check if the queue is empty)
 5. **isFull** (Check if the queue is full - applicable for arrays)
-

Implementation of Queue in C

Queues can be implemented in two ways:

1. Using **Arrays**
 2. Using **Linked Lists**
-

1. Queue Using Arrays

A queue using an array requires two pointers:

- **front** → Points to the first element of the queue.
- **rear** → Points to the last element of the queue.

Implementation:

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int queue[MAX], front = -1, rear = -1;
```

```
// Function to check if the queue is empty
```

```
int isEmpty() {
```

```
    return front == -1 || front > rear;
```

```
}
```

```
// Function to check if the queue is full
```

```
int isFull() {
```

BSDVP ACADEMY

```
        return rear == MAX - 1;
    }

// Function to enqueue (insert) an element
void enqueue(int value) {
    if (isFull()) {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1) front = 0; // Initialize front if first element
    queue[++rear] = value;
    printf("Enqueued: %d\n", value);
}

// Function to dequeue (remove) an element
int dequeue() {
    if (isEmpty()) {
        printf("Queue Underflow\n");
        return -1;
    }
    int removedElement = queue[front++];
    if (front > rear) front = rear = -1; // Reset when queue is empty
    return removedElement;
}

// Function to get the front element
int frontElement() {
    if (isEmpty()) {
        printf("Queue is empty\n");
```

BSDVP ACADEMY

```
        return -1;
    }
    return queue[front];
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    printf("Front element: %d\n", frontElement());
    printf("Dequeued element: %d\n", dequeue());
    printf("Front element: %d\n", frontElement());
    return 0;
}
```

Complexity Analysis of Array-based Queue:

- **Enqueue (Insertion)** $\rightarrow O(1)$
- **Dequeue (Deletion)** $\rightarrow O(1)$
- **Peek (Front element)** $\rightarrow O(1)$

Limitations of Array-Based Queues:

- Fixed size (cannot expand dynamically).
- After several dequeues, space is wasted unless managed properly.

2. Queue Using Linked List

A linked list-based queue uses dynamic memory allocation. It consists of **nodes** with two fields:

- **Data**
- **Pointer to the next node**

Implementation:

```
#include <stdio.h>

#include <stdlib.h>
```

BSDVP ACADEMY

```
struct Node {  
    int data;  
    struct Node* next;  
};  
  
struct Node *front = NULL, *rear = NULL;  
  
// Function to check if the queue is empty  
int isEmpty() {  
    return front == NULL;  
}  
  
// Function to enqueue (insert) an element  
void enqueue(int value) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = value;  
    newNode->next = NULL;  
  
    if (rear == NULL) {  
        front = rear = newNode;  
    } else {  
        rear->next = newNode;  
        rear = newNode;  
    }  
    printf("Enqueued: %d\n", value);  
}  
  
// Function to dequeue (remove) an element  
int dequeue() {
```


BSDVP ACADEMY

```
if (isEmpty()) {
    printf("Queue Underflow\n");
    return -1;
}

int removedData = front->data;
struct Node* temp = front;
front = front->next;

if (front == NULL) rear = NULL; // If the queue becomes empty
free(temp);
return removedData;
}

// Function to get the front element
int frontElement() {
    if (isEmpty()) {
        printf("Queue is empty\n");
        return -1;
    }
    return front->data;
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    printf("Front element: %d\n", frontElement());
    printf("Dequeued element: %d\n", dequeue());
    printf("Front element: %d\n", frontElement());
}
```

BSDVP ACADEMY

```
    return 0;
}
```

Complexity Analysis of Linked List-Based Queue:

- **Enqueue (Insertion)** $\rightarrow O(1)$
- **Dequeue (Deletion)** $\rightarrow O(1)$
- **Peek (Front element)** $\rightarrow O(1)$

Advantages of Linked List-Based Queues:

- No fixed size constraint (dynamically allocated).
- Efficient memory utilization.

Disadvantages:

- Requires extra memory for pointers.
- Slightly more complex to implement.

3. Circular Queue

A **circular queue** solves the problem of wasted space in an array-based queue by making the last index connect back to the first index.

Implementation of Circular Queue Using Arrays

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int queue[MAX], front = -1, rear = -1;
```

```
int isFull() {
    return (rear + 1) % MAX == front;
}
```

```
int isEmpty() {
    return front == -1;
}
```

BSDVP ACADEMY

```
void enqueue(int value) {
    if (isFull()) {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1) front = 0;
    rear = (rear + 1) % MAX;
    queue[rear] = value;
    printf("Enqueued: %d\n", value);
}

int dequeue() {
    if (isEmpty()) {
        printf("Queue Underflow\n");
        return -1;
    }
    int removedElement = queue[front];
    if (front == rear) front = rear = -1; // Reset queue
    else front = (front + 1) % MAX;
    return removedElement;
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    printf("Dequeued: %d\n", dequeue());
    enqueue(40);
    enqueue(50);
}
```

BSDVP ACADEMY

```
enqueue(60); // Should show overflow  
return 0;  
}
```

Comparison of Different Queue Implementations

Implementation	Memory Usage	Time Complexity (Enqueue/Dequeue)	Complexity of Checking Empty/Full
Array Queue	Fixed Size	O(1)	O(1)
Linked List Queue	Dynamic	O(1)	O(1)
Circular Queue	Fixed but efficient	O(1)	O(1)

Comparison of Linear Data Structures

Data Structure	Memory Usage	Access Time	Insertion/Deletion	Usage
Array	Fixed size, contiguous	O(1)	O(n) (shifting required)	Static data storage
Linked List	Dynamic, extra pointer memory	O(n)	O(1) at the beginning	Dynamic memory management
Stack	Contiguous (array) or dynamic (linked list)	O(n)	O(1)	Function calls, expression evaluation
Queue	Contiguous (array) or dynamic (linked list)	O(n)	O(1)	Scheduling, buffering

5. Non-Linear Data Structures in C – Trees

Introduction

A **tree** is a non-linear data structure that represents hierarchical relationships between elements. It consists of **nodes** connected by **edges**, with a single node at the top called the **root**.

Unlike arrays, linked lists, stacks, or queues, which are linear data structures, trees allow a more complex relationship among data elements, making them useful for various applications like database indexing, file systems, and Artificial Intelligence (AI).

BSDVP ACADEMY

1. Basic Terminology in Trees

Before diving into implementation, let's understand some key terms related to trees:

1. **Node** – A fundamental unit of the tree that stores data.
 2. **Root** – The topmost node in a tree.
 3. **Edge** – A connection between two nodes.
 4. **Parent Node** – The node that has child nodes.
 5. **Child Node** – The node that descends from another node.
 6. **Siblings** – Nodes that share the same parent.
 7. **Leaf Node** – A node that has no children.
 8. **Depth** – The number of edges from the root to a node.
 9. **Height** – The number of edges from a node to the deepest leaf.
 10. **Degree** – The total number of children a node has.
 11. **Subtree** – A tree formed by a node and its descendants.
-

2. Types of Trees

(a) Binary Tree

A **binary tree** is a tree in which each node has at most **two children** (left and right).

Types of Binary Trees

1. **Full Binary Tree** – Every node has either **0 or 2** children.
2. **Complete Binary Tree** – All levels are filled except possibly the last, which is filled from **left to right**.
3. **Perfect Binary Tree** – All leaf nodes are at the same level, and all internal nodes have two children.
4. **Balanced Binary Tree** – The height difference between the left and right subtrees of any node is at most 1.
5. **Degenerate (Skewed) Tree** – Every parent has only one child, making it similar to a linked list.

(b) Binary Search Tree (BST)

A **BST** is a binary tree where:

- The **left subtree** contains nodes with values **less than** the parent.
- The **right subtree** contains nodes with values **greater than** the parent.

BSDVP ACADEMY

This structure allows efficient searching, insertion, and deletion operations in **$O(\log n)$** time.

(c) AVL Tree (Self-Balancing BST)

An **AVL tree** is a self-balancing BST where the difference in heights of the left and right subtrees of any node is at most **1**.

(d) B-Trees & B+ Trees

Used in databases and file systems for efficient storage and retrieval.

3. Binary Tree Representation in C

A **binary tree** in C can be represented using **structures and pointers**.

Structure Definition

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Define a structure for a tree node
```

```
struct Node {  
    int data;  
    struct Node* left;  
    struct Node* right;  
};
```

```
// Function to create a new node
```

```
struct Node* createNode(int value) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = value;  
    newNode->left = NULL;  
    newNode->right = NULL;  
    return newNode;  
}
```

```
int main() {
```

BSDVP ACADEMY

```
struct Node* root = createNode(10);

root->left = createNode(5);

root->right = createNode(15);


printf("Root Node: %d\n", root->data);

printf("Left Child: %d\n", root->left->data);

printf("Right Child: %d\n", root->right->data);


return 0;

}
```

Explanation

1. A Node structure is defined with **data, left, and right pointers**.
2. createNode() function allocates memory for a new node.
3. A simple **binary tree** is created with a root and two children.

4. Traversal Techniques

Tree traversal is the process of visiting all nodes of a tree systematically.

(a) Inorder Traversal (Left → Root → Right)

```
void inorder(struct Node* root) {

    if (root == NULL) return;

    inorder(root->left);

    printf("%d ", root->data);

    inorder(root->right);

}
```

Output Example: 5 10 15 (for the above tree)

(b) Preorder Traversal (Root → Left → Right)

```
void preorder(struct Node* root) {

    if (root == NULL) return;

    printf("%d ", root->data);

}
```

BSDVP ACADEMY

```
preorder(root->left);  
preorder(root->right);  
}
```

Output Example: 10 5 15

(c) Postorder Traversal (Left → Right → Root)

```
void postorder(struct Node* root) {  
    if (root == NULL) return;  
    postorder(root->left);  
    postorder(root->right);  
    printf("%d ", root->data);  
}
```

Output Example: 5 15 10

5. Operations on Binary Search Tree (BST)

(a) Insertion in BST

```
struct Node* insert(struct Node* root, int value) {  
    if (root == NULL) return createNode(value);  
    if (value < root->data)  
        root->left = insert(root->left, value);  
    else  
        root->right = insert(root->right, value);  
    return root;  
}
```

(b) Searching in BST

```
struct Node* search(struct Node* root, int key) {  
    if (root == NULL || root->data == key) return root;  
    if (key < root->data) return search(root->left, key);  
    return search(root->right, key);  
}
```


BSDVP ACADEMY

(c) Deletion in BST

```
struct Node* findMin(struct Node* root) {  
    while (root->left != NULL)  
        root = root->left;  
    return root;  
}  
  
struct Node* deleteNode(struct Node* root, int key) {  
    if (root == NULL) return root;  
    if (key < root->data)  
        root->left = deleteNode(root->left, key);  
    else if (key > root->data)  
        root->right = deleteNode(root->right, key);  
    else {  
        if (root->left == NULL) return root->right;  
        else if (root->right == NULL) return root->left;  
        struct Node* temp = findMin(root->right);  
        root->data = temp->data;  
        root->right = deleteNode(root->right, temp->data);  
    }  
    return root;  
}
```

6. Applications of Trees

- **Binary Search Trees (BSTs)** – Used in searching and sorting algorithms.
- **Heap Trees** – Used in **priority queues**.
- **Expression Trees** – Used in **compilers and calculators**.
- **Decision Trees** – Used in **machine learning**.
- **B-Trees & B+ Trees** – Used in **database indexing**.

BSDVP ACADEMY

 ONE APP TO LEARN EVERYTHING! 

Looking for the ultimate learning companion? Look no further! BSDVP Academy is your go-to app for all your educational needs!

☀️ Download BSDVP Academy NOW from the Google Play Store and unlock a world of knowledge at your fingertips! ☀️

Key Features:

- ✓ Comprehensive Study Materials & Short Notes
- ✓ ECET Test Series & Guidance
- ✓ Exam Updates & Results
- ✓ GATE Exam Support
- ✓ Job Alerts for Diploma Holders
- ✓ Perfect Your Resume
- ✓ Complete MS Office Package
- ✓ Stay Ahead with Education Updates

Why BSDVP Academy?

- Convenient: All-in-one app to cater to all your educational needs.
- Comprehensive: Covers everything from exams to job updates.
- Reliable: Stay up-to-date with the latest in your field.



Download now and take the first step towards a smarter future!

👉 BSDVP Academy

Android App– <https://play.google.com/store/apps/details?id=com.szlwfh.ijkyoi>

Apple Iphone App- <https://apps.apple.com/in/app/my-coaching-by-appx/id1662307591>



Available on Google Play Store.